

1. Worst-case complexity derivation

In the worst-case, the pivot is always the smallest element, so one partition is empty and the other contains $(n-1)$ elements. This gives the recurrence

$$T(n) = T(n-1) + O(n)$$

Straighten the recurrence results in:

$$T(n) = O(n) + O(n-1) + \dots + O(1) = O(n^2)$$

2.

* Example of a vector of 16 elements causing worst-case:
considers the sorted vector:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]

Worst-cases:

- First call: pivot = 1 ← the smallest left () right (2, 3, 4, ..., 16)
 - Second call: pivot = 2 left () right (3, 4, ..., 16)
 - Third call: pivot = 3 left () right (...)
- ,
- |
- |

3. Measurement results

We ran quicksort in worst-case inputs (i.e., sorted arrays) for increasing sizes. The measured execution times increase roughly quadratically with n .

4. Plot discussion

We ran quicksort in worst-case inputs (i.e., sorted arrays) for increasing sizes. The measured execution times increase roughly quadratically with n . The plot shows a clear quadratic trend in execution time, matching the $O(n^2)$ worst-case complexity derived earlier.

